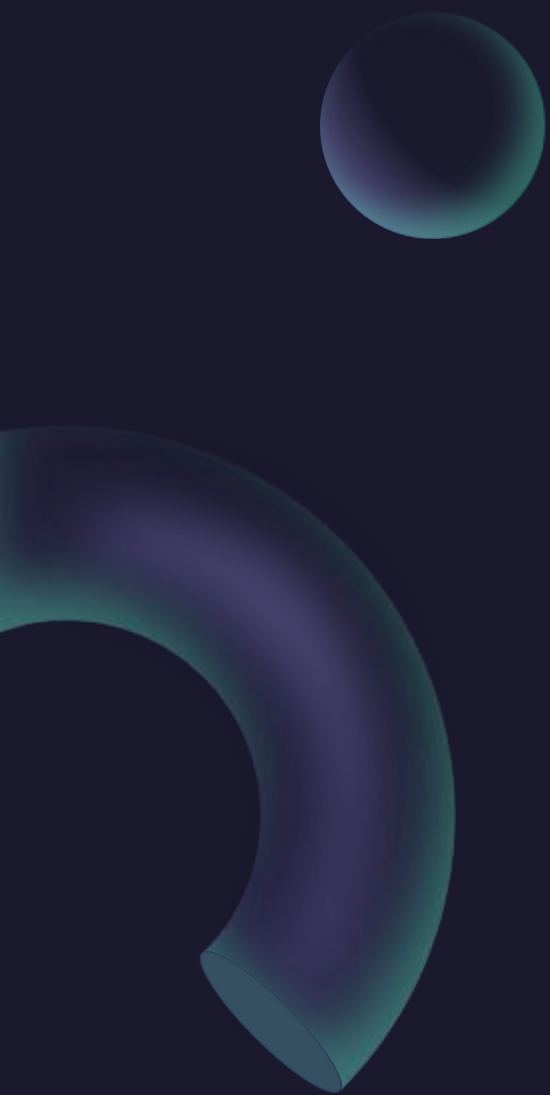
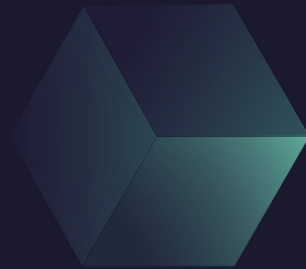


A decorative graphic on the left side of the slide. It features a small, dark blue sphere with a teal gradient at the top left, and a larger, dark blue ring with a teal gradient below it, partially visible on the left edge.

SOFTWARE ARCHITECTURE

High Level Overview

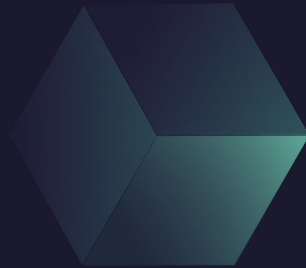
Software Architecture



- Refers to the fundamental structures of a software system and the discipline of creating such structures.
- High-level organization of a system's components, their relationships,
- Principles governing their design and evolution.
- There is no GOLD standard and it evolves

Key Concepts

- Architectural Styles
- Quality Attributes
- Architectural Patterns



Architectural Styles

- Common patterns for system organization
 - Monolithic: Single-tiered application where all components are interconnected
 - Client-Server: Separates providers (servers) from requesters (clients)
 - Microservices: Collection of small, independent services
 - Event-Driven: Components communicate through events
 - Layered (n-tier): Separation of concerns into distinct layers
 - Service-Oriented (SOA): Services communicate over a network



Quality Attributes

- Non-functional requirements that influence architecture:
 - ✓ Performance: Response time, throughput
 - ✓ Scalability: Ability to handle growth
 - ✓ Reliability: System uptime and fault tolerance
 - ✓ Security: Protection against threats
 - ✓ Maintainability: Ease of modification
 - ✓ Easy Deployment: Ease of deployment



Architectural Patterns

✓ Some Common Patterns

- ✓ Model-View-Controller (MVC): Separates data, presentation, and control
- ✓ Publish-Subscribe: Decouples message senders and receivers
- ✓ Repository: Centralized storage for domain objects
- ✓ Event Driven: Event bus is used for communication



Design Principles

- ✓ Separation of Concerns: Divide system into distinct features
- ✓ Single Responsibility: Each component should have one purpose
- ✓ Least Knowledge: Components should know little about others
- ✓ Don't Repeat Yourself (DRY): Avoid duplication
- ✓ KISS (Keep It Simple): Prefer simple solutions



Separation of Concerns (SoC)

- Divide a system into distinct modules where each module handles **a specific functionality**.
 - **Reduces complexity,**
 - **Improves maintainability**
 - **Allows independent development.**
 - **Example:**
 - **Frontend (UI):** Handles user interaction.
 - **Backend (API):** Manages business logic & data.
 - **Database:** Stores and retrieves data.
 - **Each part has a clear responsibility without overlapping.**



Separation of Concerns (SoC) - Example

```
• // routes/patients.js

• router.get('/:id', async (req, res) => {

•   const patient = await db.query('SELECT * FROM patients WHERE id = $1', [req.params.id]);

•   res.json(patient);

• });
```



Separation of Concerns (SoC) - Example

- ```
// controllers/patientController.js
const PatientService = require('../services/patientService');

exports.getPatient = async (req, res) => {
 const patient = await PatientService.getById(req.params.id);
 res.json(patient);
};

// services/patientService.js
const PatientRepository = require('../repositories/patientRepository');

exports.getById = async (id) => {
 return await PatientRepository.findById(id);
};

// repositories/patientRepository.js
exports.findById = async (id) => {
 return await db.query('SELECT * FROM patients WHERE id = $1', [id]);
};
```

# Single Responsibility

- A class/module/function should have only **one reason to change** (i.e., one job).
  - Code easier to test,
  - Modify, and reuse.
- Example:
  -  Bad: A class that handles both user authentication and sending emails.
  -  Good:
    - AuthService → Only handles login/logout.
    - EmailService → Only sends emails.
    - If email logic changes, we don't touch AuthService.



# Single Responsibility(Example)

-  **Bad (One class does everything)**

- ```
class MedicationManager {
```
- ```
 async checkInteractions(meds) {
```
- ```
        // Check drug-drug interactions
```
- ```
 }
```
- 
- ```
    async sendAlerts(patientId) {
```
- ```
 // Send SMS/email alerts
```
- ```
        // Log to database
```
- ```
 }
```

# Single Responsibility(Example)

-  **Good (Split services)**

- ```
class InteractionChecker {  
    async check(meds) { /* ... */ }  
}
```

```
class AlertSender {    //Can also create ISender interface and create 2 concrete implementation  
    //Email/SMS
```

- ```
 async sms(patientId) { /* ... */ }
 async email(patientId) { /* ... */ }
}
```

```
class AlertLogger {
 async log(patientId) { /* ... */ }
}
```

# Least Knowledge

- A component should only talk to its immediate friends (avoid deep object chains).
- Reduces coupling between components.



# Least Knowledge (example)

- **✗ Bad (Deep property chain)**

- `// BillingService.js`
- `const getInsurance = (patient) => {`
- `return patient.insurance.provider.plan.copay; // 🤪`
- `};`



# Least Knowledge (example)

-  Good (Delegate method)

- ```
// Patient.js
class Patient {
  getCopay() {
    return this.insurance.provider.plan.copay;
  }
}
```

```
// BillingService.js
const getInsurance = (patient) => {
  return patient.getCopay(); // Clean!
};
```



Don't Repeat Yourself (DRY)

- Avoid duplicating code; reuse logic via functions/modules.
 - Easier maintenance (change once, update everywhere).

-



Don't Repeat Yourself (DRY)[Example]

✗ Bad (Repeated auth check)

```
// routes/appointments.js
```

```
router.get('/', async (req, res) => {  
  if (!req.user.isDoctor) return res.status(403).end();  
  
  // ...  
});
```

```
// routes/patients.js
```

```
router.post('/', async (req, res) => {  
  if (!req.user.isDoctor) return res.status(403).end();
```

Don't Repeat Yourself (DRY)[Example]

- `// middleware/auth.js`
- `exports.doctorOnly = (req, res, next) => {`
- `if (!req.user.isDoctor) return res.status(403).end();`
- `next();`
- `};`
-
- `// routes/appointments.js`
- `router.get('/', doctorOnly, async (req, res) => { /* ... */ });`
-

Keep It Simple, Stupid

- Prefer simple, straightforward solutions over complex ones.
- Complexity introduces bugs and maintenance headaches.
- Write the dumbest code that could possibly work, then refactor only when needed.



Keep It Simple, Stupid – Example

-  Overcomplicated Version (*Using unnecessary classes and abstractions*)
- `// 1. Abstract "CheckInStrategy" interface (Why?)`
- `class CheckInStrategy {`
- `execute(patientId) { throw new Error('Not implemented!'); }`
- `}`
-
- `// 2. Concrete strategy implementation`
- `class StandardCheckIn extends CheckInStrategy {`
- `async execute(patientId) {`
- `// ...10 lines of check-in logic...`
- `}`
- `}`

Keep It Simple, Stupid – Example

-  Overcomplicated (Continue...)

```
// 4. Main service class
```

```
class CheckInService {  
  
    constructor() {  
  
        this.repository = new PatientRepository();  
  
    }  
  
    async checkIn(patientId, type = 'standard') {  
  
        const strategy = CheckInFactory.create(type);  
  
        return strategy.execute(patientId);  
  
    }  
}
```

```
// 5. API endpoint
```

Keep It Simple, Stupid – Example

✓ KISS Version

```
• const db = require('./db'); // Simple database client  
  
•  
  
• // 1. Single API endpoint with direct logic  
  
• router.post('/checkin', async (req, res) => {  
  
•   try {  
  
•     // Update patient status in database  
  
•     await db.query(  
  
•       `UPDATE patients  
  
•         SET status = 'checked_in', last_checkin = NOW()  
  
•         WHERE id = $1`,  
  
•       [req.body.patientId]
```

Keep It Simple, Stupid – Example

-  **Over-engineered:** // Using a complex state management library for a small app.
-  **Simple:** // Using React's built-in `useState` for local state.



Key Takeaways

Principle	Goal	Pitfall to Avoid
Separation of Concerns	Modular design	Mixing UI & business logic
Single Responsibility	One job per component	"God objects" doing everything
Least Knowledge	Loose coupling	Deep object traversal
DRY	Code reuse	Copy-pasting logic
KISS	Simplicity	Over-engineering



Modern Trends

- Containerization : Docker , Kubernetes
- Serverless : No server
- Cloud Native: AWS, GCP
- Edge Computing : Processing data closer to the source

